

CS240 Algorithm Design and Analysis

Spring 2026

Problem Set 4

Due: 23:59, June 10, 2026

1. Please write your solutions in English.
2. Submit your solutions to **Gradescope**. When submitting, match your solutions to the problems correctly.
3. If you want to submit a handwritten version, scan it clearly.
4. Up to 2 late days are allowed for homework submissions. Assignments submitted within the first 24 hours after the deadline will incur a 10% penalty; those submitted within 48 hours will incur a 20% penalty. Submissions more than 48 hours late will not be accepted unless there is a valid reason, such as a medical or family emergency.
5. You are required to follow ShanghaiTech's academic honesty policies. You are allowed to discuss problems with other students, but you must write up your solutions by yourselves. You are not allowed to copy materials from other students or from online or published resources. Violating academic honesty can result in serious penalties.

Problem 1: Lazy Disk-Write Queue

A cloud storage system manages n data pages, numbered $1, 2, \dots, n$. After a page is modified, it becomes a *dirty page*, meaning that the page has unsaved contents that have not yet been written back to disk.

The system maintains a Boolean array `dirty` $[1 \dots n]$, where `dirty` $[i] = \text{true}$ means that page i has unsaved modifications. The system also maintains a FIFO queue Q , whose entries are page IDs requested for write-back.

The system supports the following two operations:

- `MODIFY`(i): set `dirty` $[i] \leftarrow \text{true}$, and append i to the rear of Q . Even if `dirty` $[i]$ is already `true`, this operation still appends another copy of i to Q .
- `FLUSHONE`(): repeatedly remove the front entry j from Q . If `dirty` $[j] = \text{false}$, then this queue entry is stale and is discarded. If `dirty` $[j] = \text{true}$, then page j is written back, `dirty` $[j] \leftarrow \text{false}$, and j is returned. If Q becomes empty, the operation returns 0.

Each queue insertion, queue deletion, array access, and array assignment has cost 1. Writing back one dirty page also has cost 1.

Answer the following questions:

- (a) Construct an operation sequence in which one call to `FLUSHONE`() has cost $\Theta(m)$, where m is the number of operations performed so far.
- (b) Prove that the total running time of any sequence of m operations is $O(m)$. Your proof should also explain why duplicate page IDs in Q do not break the bound.
- (c) Give a potential function and use the potential method to prove that both `MODIFY` and `FLUSHONE` have amortized running time $O(1)$.

Solution:

The key point is that the queue is processed lazily. A call to `FLUSHONE` may skip many stale entries, so its single-operation worst-case cost can be large. However, every queue entry is inserted once and removed at most once.

(a) Single-operation worst case. Execute k operations `MODIFY`(1) in a row. Then Q contains k copies of page ID 1, and `dirty` $[1] = \text{true}$. The first call to `FLUSHONE`() removes the first copy of 1, writes back page 1, and sets `dirty` $[1] \leftarrow \text{false}$. Now the queue still contains $k-1$ copies of 1, but all of them

are stale. A second call to `FLUSHONE()` removes all these $k - 1$ stale entries before returning 0. Therefore, this call costs $\Theta(k)$. At this point the number of operations performed so far is $m = k + 2$, so the cost is $\Theta(m)$.

(b) Aggregate analysis and duplicate IDs. Each `MODIFY` operation inserts one entry into Q . Each inserted queue entry can be removed from Q at most once. Thus, over any sequence of m operations, the total number of queue insertions is $O(m)$, and the total number of queue deletions is also $O(m)$. Apart from queue operations, each external operation performs only a constant number of array accesses, array assignments, and at most one write-back. Hence the total running time is $O(m)$.

Duplicate page IDs do not cause a problem because each duplicate entry is created by a distinct `MODIFY` operation. Even if many entries have the same page ID, each queue entry is still removed at most once. Therefore duplicates only add cost proportional to the number of previous `MODIFY` operations.

(c) Potential method. Let the potential be

$$\Phi = C|Q|,$$

where $|Q|$ is the current number of entries in the queue and C is a sufficiently large positive constant. The potential is always nonnegative, and initially $\Phi = 0$.

For `MODIFY( $i$ )`, the actual cost is $O(1)$, and $|Q|$ increases by 1. Thus $\Delta\Phi = C$, so the amortized cost is $O(1)$.

For `FLUSHONE()`, suppose the operation deletes k queue entries in total. Its actual cost is $O(k) + O(1)$, and the queue size decreases by k , so the potential decreases by Ck . By choosing C large enough, this potential drop pays for all deleted stale entries. Hence the amortized cost of `FLUSHONE` is $O(1)$. If Q is already empty, both the actual and amortized costs are $O(1)$.

Problem 2: Triangular Calibration Monitor

A scientific monitoring device continuously receives samples. The operation `INSERTSAMPLE( $x$ )` inserts one new sample and has base cost 1. Let N be the total number of samples inserted so far. Initially, $N = 0$.

Whenever N becomes a triangular number, the device automatically performs one calibration. Define $T_0 = 0$, and for $q \geq 1$,

$$T_q = 1 + 2 + \cdots + q = \frac{q(q+1)}{2}.$$

When the q -th calibration is triggered, its cost is exactly q . For example, after the 1st insertion the calibration cost is 1, after the 3rd insertion the calibration cost is 2, after the 6th insertion the calibration cost is 3, and after the 10th insertion the calibration cost is 4.

Answer the following questions:

- (a) Given n , how many calibrations occur during the first n insertions?
- (b) Use aggregate analysis to prove that the total cost of the first n insertions is $O(n)$.
- (c) Use the accounting method to give a constant amortized charge for each insertion.

Solution:

This is a triggered batch-processing model. Insertions are cheap, but some insertions trigger calibrations. The important observation is that the q -th calibration has cost q , while the gap between consecutive triangular numbers grows linearly.

(a) Number of calibrations. The number of calibrations during the first n insertions is the largest integer q such that $T_q \leq n$. That is,

$$\frac{q(q+1)}{2} \leq n.$$

Solving this inequality gives

$$q = \left\lfloor \frac{\sqrt{8n+1}-1}{2} \right\rfloor.$$

(b) Aggregate analysis. The base cost of the first n insertions is n . Suppose q calibrations occur. Their total cost is

$$1 + 2 + \cdots + q = T_q \leq n.$$

Therefore, the total cost is at most

$$n + T_q \leq 2n,$$

which is $O(n)$. Hence the amortized cost per insertion is $O(1)$.

(c) Accounting method. Charge each INSERTSAMPLE operation 2 units. One unit pays for the insertion itself, and the other unit is stored as credit.

After the q -th calibration, the next calibration is the $(q + 1)$ -st calibration, whose cost is $q + 1$. From just after the q -th calibration until the insertion that triggers the $(q + 1)$ -st calibration, there are exactly $q + 1$ insertions. Each of them stores one unit of credit, so these insertions store exactly $q + 1$ units, enough to pay for the next calibration. Thus the credit never becomes negative, and the amortized cost is 2 per insertion.

Problem 3: Archive with Tombstone Compaction

An online news platform stores articles in an array. Each array entry is either a live article or a tombstone. A tombstone means that the article has been deleted, but its array position has not yet been physically removed.

The system also maintains a mapping from article IDs to array positions. Therefore, checking whether an ID corresponds to a live article and marking it as a tombstone can both be done in $O(1)$ time. During compaction, the system also updates this mapping. Updating the mapping for a copied live article is included in the copying cost of that live article.

Users can perform the following two external operations:

- **ADD(*article*)**: append the new article as a live article at the end of the array, and record the mapping from its ID to its array position.
- **REMOVE(*id*)**: if the article with ID *id* is still live, mark its entry as a tombstone. If this ID does not exist or is no longer live, perform only one lookup and return.

The system also has the following internal cleanup procedure:

- **COMPACT()**: scan the entire array, copy all live articles into a new array, discard all tombstones, and update the ID-to-position mapping.

COMPACT() is not directly called by users. It is automatically triggered by the system after a **REMOVE** operation.

System rule: after each **REMOVE**, if the number of tombstones is strictly greater than the number of live articles, the system automatically executes **COMPACT()**.

Assume that appending an article costs 1, marking a tombstone costs 1, and scanning or copying one array entry costs 1.

Answer the following questions:

- (a) Explain why a single **REMOVE** operation can have worst-case cost $\Theta(n)$, where n is the array length immediately before the **COMPACT** operation triggered by this **REMOVE**.
- (b) Prove that the total cost of any m external operations is $O(m)$.
- (c) Give a potential function based on the number of tombstones and prove an amortized $O(1)$ bound.

Solution:

This is a lazy-deletion and batch-compaction model. Deletions do not immediately move array entries. Instead, they leave tombstones, and the system performs a global compaction only after enough tombstones have accumulated.

(a) Single-operation worst case. A REMOVE operation may make the number of tombstones strictly greater than the number of live articles. Then the operation triggers COMPACT(). If the array length immediately before compaction is n , then compaction scans the whole array and copies the live articles, so this single REMOVE operation can cost $\Theta(n)$.

(b) Aggregate analysis. Every tombstone is created by one effective REMOVE operation and is discarded by the next compaction that includes it.

Consider one compaction. Let D be the number of tombstones and L the number of live articles just before compaction. Since compaction is triggered only when

$$D > L,$$

the current array length satisfies

$$D + L < 2D.$$

The compaction scans $D + L$ entries and copies L live articles. Therefore, its cost is at most

$$(D + L) + L = D + 2L < 3D.$$

Thus the cost of this compaction is $O(D)$. These D tombstones were created by D effective REMOVE operations since the previous compaction, including the triggering REMOVE. Therefore, the compaction cost can be charged to those removals.

Over the whole operation sequence, the basic costs of ADD and REMOVE are $O(m)$, and the total cost of all compactions is also $O(m)$. Hence the total cost of any m external operations is $O(m)$.

(c) Potential method. Let D be the current number of tombstones. Define

$$\Phi = CD,$$

where C is a sufficiently large positive constant, for example $C = 4$. The potential is nonnegative and is initially 0.

An ADD operation does not change D . Its actual cost is $O(1)$, and the potential does not increase, so its amortized cost is $O(1)$. An ineffective REMOVE operation also does not change D , so its amortized cost is $O(1)$.

An effective REMOVE that does not trigger compaction increases D by 1. Its actual cost is $O(1)$, and the potential increases by C , so its amortized cost is $O(1)$.

Now consider an effective REMOVE that triggers compaction. Let D and L be the number of tombstones and live articles after this removal but before compaction. Then $D > L$, so the compaction cost is less than $3D$. Before this REMOVE, the number of tombstones was $D - 1$. After compaction, the number of tombstones becomes 0. Thus

$$\Delta\Phi = 0 - C(D - 1).$$

For $C \geq 4$, the potential drop pays for the compaction cost up to a constant. Therefore the amortized cost of the triggering REMOVE is $O(1)$. The condition $D > L$ is crucial because it guarantees that compaction is performed only after many tombstones have accumulated.

Problem 4: Random Audit without Replacement

A course has n homework submissions, and exactly r of them have formatting violations. The instructor randomly selects s distinct submissions for manual audit, using sampling without replacement. Assume $0 \leq r \leq n$ and $0 \leq s \leq n$.

Let X be the number of selected submissions that have formatting violations.

Answer the following questions:

- (a) Define suitable indicator random variables and compute $\mathbb{E}[X]$.
- (b) Compute the probability that the audit finds no formatting violation.
- (c) Give a simple exponential upper bound for the probability in part (b), and use it to give a sufficient condition on s so that the instructor finds at least one violation with probability at least $1 - \delta$, where $r > 0$ and $0 < \delta < 1$.

Solution:

This problem uses expectation under sampling without replacement. The main idea is to define an indicator random variable for each bad submission and then apply linearity of expectation.

(a) Indicator random variables and expectation. Let the set of submissions with formatting violations be

$$B = \{b_1, b_2, \dots, b_r\}.$$

For each bad submission b_i , define

$$X_i = \begin{cases} 1, & \text{if } b_i \text{ is selected,} \\ 0, & \text{otherwise.} \end{cases}$$

Then

$$X = X_1 + X_2 + \dots + X_r.$$

For any fixed submission, the probability that it is selected is s/n . Hence

$$\mathbb{E}[X_i] = \frac{s}{n}.$$

By linearity of expectation,

$$\mathbb{E}[X] = \sum_{i=1}^r \mathbb{E}[X_i] = r \cdot \frac{s}{n} = \frac{rs}{n}.$$

The variables X_i are not independent, but linearity of expectation does not require independence.

(b) Probability of finding no violation. Finding no violation means that all s selected submissions come from the $n - r$ clean submissions. Therefore,

$$\Pr[X = 0] = \frac{\binom{n-r}{s}}{\binom{n}{s}}.$$

If $s > n - r$, then we interpret $\binom{n-r}{s} = 0$, so the probability is 0.

(c) Exponential upper bound and sufficient sample size. For $s \leq n - r$, we can write

$$\Pr[X = 0] = \prod_{i=0}^{s-1} \frac{n - r - i}{n - i} \leq \left(1 - \frac{r}{n}\right)^s \leq e^{-rs/n}.$$

The same upper bound also holds trivially when $s > n - r$, since the left-hand side is 0.

It is enough to require

$$e^{-rs/n} \leq \delta.$$

Solving this inequality gives the sufficient condition

$$s \geq \frac{n}{r} \ln \frac{1}{\delta}.$$

Since s is an integer, one may take

$$s = \left\lceil \frac{n}{r} \ln \frac{1}{\delta} \right\rceil$$

whenever this value is at most n . If this value is larger than n , the exponential-bound argument cannot be satisfied with $s \leq n$; however, choosing $s = n$ audits all submissions, so when $r > 0$, the failure probability is exactly 0.

Problem 5: One-Round Random-Priority Conflict Filter

A system has a set of tasks. Some pairs of tasks conflict and cannot be selected simultaneously. This conflict relation is represented by a simple undirected graph $G = (V, E)$: each vertex is a task, and each edge means that the two corresponding tasks conflict.

The system performs the following randomized filtering step:

1. Each vertex v independently chooses a priority $p(v)$ uniformly at random from $[0, 1]$.
2. Vertex v is selected if its priority is smaller than the priority of every neighbor of v . Here a smaller numerical priority means a higher priority.

Because the priorities are drawn from a continuous distribution, all priorities are distinct with probability 1. Let S be the set of selected vertices.

Answer the following questions:

- (a) Prove that S is always an independent set.
- (b) For a fixed vertex v , let $\deg(v)$ be its degree. What is the probability that v is selected?
- (c) Let $X = |S|$. Use indicator random variables to compute $\mathbb{E}[X]$, and simplify the answer when G is d -regular.

Solution:

This is a random-priority local-minimum rule on a graph. A vertex is selected exactly when it has the smallest priority among itself and all of its neighbors.

(a) Independent set. Suppose, for contradiction, that two adjacent vertices u and v are both selected. Since u is selected and v is its neighbor, we must have

$$p(u) < p(v).$$

Since v is selected and u is its neighbor, we must also have

$$p(v) < p(u).$$

This is impossible. Therefore no two adjacent vertices can both be selected, and S is an independent set.

(b) Selection probability of a fixed vertex. Vertex v is selected exactly when it has the smallest priority among the set

$$\{v\} \cup N(v),$$

whose size is $\deg(v) + 1$. By symmetry, each vertex in this set is equally likely to have the smallest priority. Therefore,

$$\Pr[v \in S] = \frac{1}{\deg(v) + 1}.$$

If v is isolated, then $\deg(v) = 0$, and the formula correctly gives probability 1.

(c) Expected size of S . For each vertex $v \in V$, define the indicator random variable

$$X_v = \begin{cases} 1, & \text{if } v \in S, \\ 0, & \text{otherwise.} \end{cases}$$

Then

$$X = |S| = \sum_{v \in V} X_v.$$

By linearity of expectation,

$$\mathbb{E}[X] = \sum_{v \in V} \mathbb{E}[X_v] = \sum_{v \in V} \Pr[v \in S] = \sum_{v \in V} \frac{1}{\deg(v) + 1}.$$

Linearity of expectation does not require the selected events to be mutually independent. In fact, adjacent vertices cannot both be selected.

If G is d -regular and $|V| = n$, then every vertex has degree d . Hence

$$\mathbb{E}[|S|] = \frac{n}{d + 1}.$$